
Algorithm Engineering

Esercizi di Teoria — testi e risoluzioni

Tutte le richieste della parte di *teoria* dei compitini e degli appelli 2020–2025, organizzate per argomento.

Parte I: i testi degli esercizi. **Parte II:** le risoluzioni dettagliate.

*Ogni testo ha un identificatore (es. [5.1]); la sua risoluzione nella Parte II ha lo stesso numero ([5.1]). Accanto a ogni testo sono indicati gli appelli in cui è uscito. Le risoluzioni sono elaborate a partire dalla dispensa del corso e dal testo Pearls of Algorithm Engineering (Ferragina). **Nota:** le soluzioni ufficiali del docente esistono solo come scansioni manoscritte della parte di esercizi pratici e non sono trascrivibili; per la teoria non esistono soluzioni ufficiali in formato testo, quindi le risoluzioni qui riportate sono ricostruite fedelmente dalle fonti del corso.*

Indice

I	Testi degli esercizi di teoria	2
1	Modelli & complessità I/O	3
2	Ordinamento	4
3	Random Sampling	5
4	Set Intersection	6
5	Hashing, Skip List, Treap	7
6	Prefix Search	8
7	Substring Search	9
8	Integer Coding	10
9	Statistical Coding	11
10	Strutture dati compresse	12
II	Risoluzioni dettagliate	13
11	Modelli & complessità I/O	14
12	Ordinamento	15
13	Random Sampling	17
14	Set Intersection	18
15	Hashing, Skip List, Treap	19
16	Prefix Search	20
17	Substring Search	21
18	Integer Coding	22
19	Statistical Coding	23
20	Strutture dati compresse	24

Part I

Testi degli esercizi di teoria

1 Modelli & complessità I/O

Elenco testi — Modelli e bound I/O

- **[1.1]** Enuncia la complessità I/O del multi-way MergeSort in funzione di M, B, N . (27/05/24, 09/02/24)
- **[1.2]** Enuncia i bound (upper bound): ordinare n item atomici in un modello a due livelli con memoria interna M , pagina B , 1 disco; permutarli nello stesso modello con 1 disco; ordinarli con D dischi usando il *disk striping*. (10/02/23)
- **[1.3]** Enuncia la complessità I/O del multi-way MergeSort in funzione di M, B, N ; descrivi la tecnica del *disk striping* e la complessità I/O del multi-way MergeSort su D dischi usando il disk striping. (09/02/24)
- **[1.4]** Enuncia la complessità I/O di ordinare n item con memoria M , pagina B e D dischi; confrontala con il bound ottimo e discuti le differenze. (10/01/25)
- **[1.5]** Scrivi (senza dimostrazione) la complessità I/O del multi-way MergeSort con 1 disco e poi con D dischi usando il disk striping. (04/02/25)
- **[1.6]** Descrivi il disk striping ed enuncia il bound I/O di MergeSort quando lo si usa per ordinare n item in una memoria a due livelli con D dischi, memoria M , pagina B . (03/06/25)

2 Ordinamento

Elenco testi — Snow Plow, Bounded QuickSort, RadixSort, stringhe

- **[2.1]** Dimostra che la lunghezza attesa della sequenza ordinata prodotta da Snow Plow è $2M$; e quanto vale se la probabilità di finire nell’“unsorted bucket” è $\frac{1}{4}$ invece di $\frac{1}{2}$? (15/01/24)
- **[2.2]** Come **[2.1]**, ma con probabilità $\frac{1}{8}$ invece di $\frac{1}{2}$. (21/06/24)
- **[2.3]** Enuncia e dimostra il lower bound al tempo per ordinare n stringhe di lunghezza totale N su alfabeto di taglia s ; scrivi lo pseudocodice del Multikey QuickSort e dimostra che è time-optimal; dimostra la complessità del RadixSort ottimo su n stringhe binarie di lunghezza totale N . (05/06/23)
- **[2.4]** Mostra lo pseudocodice del Multikey QuickSort, enuncia e dimostra la sua complessità temporale. (07/09/23)
- **[2.5]** Scrivi lo pseudocodice e dimostra la complessità del Multikey QuickSort sul dataset S (n stringhe, lunghezza totale N , alfabeto a); dimostra il lower bound per ordinare S . (10/02/23)
- **[2.6]** Abbozza l’algoritmo LSD RadixSort, enuncia la complessità e dimostra che è corretto. (16/01/23)
- **[2.7]** Descrivi l’algoritmo LSD per ordinare n item di b bit ciascuno. (11/07/24)
- **[2.8]** Enuncia e dimostra la complessità temporale dell’LSD RadixSort per n chiavi di b bit ciascuna; dimostra la correttezza dell’ordinamento (caso stringhe binarie). (10/01/25, 23/06/25)
- **[2.9]** Scrivi e commenta lo pseudocodice del Bounded QuickSort (quello che garantisce uno stack di taglia $O(\log n)$ / $O(\log n)$ chiamate ricorsive per ogni input). (10/01/25, 04/02/25, 23/06/25)

3 Random Sampling

Elenco testi — Reservoir Sampling

- **[3.1]** Data una sequenza S di n item (n ignoto) e un intero m , mostra lo pseudocodice del Reservoir Sampling e dimostrane la correttezza (probabilità m/n di campionare ogni item). (02/11/23)
- **[3.2]** Con $m < n/2$: pseudocodice del Reservoir Sampling e prova della correttezza (campionamento uniforme). (11/07/24)
- **[3.3]** Scrivi (solo) lo pseudocodice del Reservoir Sampling. (06/03/25, 03/06/25)

4 Set Intersection

Elenco testi — intersezione di liste ordinate

- [4.1] Date due liste ordinate L_1, L_2 di lunghezze n e m : descrivi l'algoritmo “two-level scan” per l'intersezione; enuncia e dimostra la sua complessità. (21/06/24)
- [4.2] Descrivi l'algoritmo “mutual-partition” per l'intersezione di L_1, L_2 . (11/07/24)
- [4.3] Descrivi il “doubling algorithm” per l'intersezione, enuncia e dimostra la complessità. (15/01/24)
- [4.4] Descrivi l'algoritmo “two-level memory” per l'intersezione; enuncia e dimostra la complessità. (09/02/24)
- [4.5] Mostra e dimostra la complessità del two-level intersection algorithm. (11/11/24, 15/07/25)
- [4.6] Scrivi lo pseudocodice per intersecare due liste L_1, L_2 codificate con Elias–Fano usando le primitive ACCESS e GEQ; valuta la complessità. (14/01/22)

5 Hashing, Skip List, Treap

Elenco testi — hashing universale, cuckoo, Bloom, Skip List, Treap

- **[5.1]** Definisci una classe di funzioni hash universali; fornisci un esempio e dimostrane l'universalità. (16/01/23, 07/09/23, 02/11/23, 10/01/25)
- **[5.2]** Definisci una classe universale e mostra che con il chaining ottiene lunghezza attesa $O(1)$ per catena quando $m = O(n)$. (06/11/23)
- **[5.3]** Enuncia e dimostra il teorema sulla probabilità che il cammino minimo fra due nodi i, j del grafo cuckoo abbia lunghezza L (per tabella di m/r posizioni, n chiavi, $r \geq 2cn$, forma c^{-L}/r); commenta l'impatto sul load factor. (10/02/23, 03/06/25)
- **[5.4]** Calcola la probabilità che una posizione del bit-array del Bloom Filter sia 0 (dimensione m , n chiavi) e deriva/dimostra la probabilità d'errore complessiva; oppure dimostra che tale probabilità è $\frac{1}{2}$ con numero ottimo di funzioni hash. (11/11/20, 27/05/24, 11/11/24, 15/07/25)
- **[5.5]** Definisci un Bloom Filter su n chiavi e le operazioni supportate; enuncia e dimostra il risultato sul falso positivo. (24/07/23)
- **[5.6]** Dati due insiemi A, B su due server, calcola la loro intersezione con un Bloom Filter e un solo round di comunicazione (con possibili errori). (05/06/23)
- **[5.7]** Dimostra che l'altezza (profondità) media di una Skip List con n nodi è $O(\log n)$. (11/11/24, 04/02/25)
- **[5.8]** Definisci la struttura Treap su n coppie $\langle \text{chiave}, \text{priorità} \rangle$ con priorità minima nella radice; descrivi come si implementa lo *split* su una chiave K non presente. (10/02/23, 23/06/25)

6 Prefix Search

Elenco testi — Patricia/compacted trie, front coding, interpolation

- **[6.1]** Dato un Patricia Trie su un dizionario S di n stringhe, descrivi come cercare la posizione lessicografica di $P[1, p]$ in S ; enuncia la complessità in tempo e I/O. (24/07/23)
- **[6.2]** Descrivi la differenza fra Compacted Trie e Patricia Trie in termini di spazio e tempo di query, in funzione di n , lunghezza totale N e lunghezza del pattern P . (06/03/25)
- **[6.3]** Qual è la differenza principale fra Compacted Trie e Patricia Trie, e come l'abbiamo sfruttata per costruire un indice a due livelli efficiente per un insieme di stringhe. (03/06/25)
- **[6.4]** Dato un dizionario D di n stringhe di lunghezza totale N , discuti almeno 3 soluzioni di memorizzazione con relativo spazio e costo tempo/IO di $\text{ACCESS}(i)$. (27/05/24)
- **[6.5]** Descrivi come usare una compacted trie (con fan-out di ogni nodo in hash table) per ordinare n stringhe di lunghezza totale L ; commenta tempo e spazio. (10/07/20)

7 Substring Search

Elenco testi — Suffix Array, LCP, COUNT, costruzione, Suffix Tree

- **[7.1]** Definisci formalmente il suffix array SA di $T[1, n]$ e il corrispondente array LCP. (16/01/23, 02/11/23)
- **[7.2]** Come **[7.1]**, più: descrivi come cercare $P[1, p]$ in SA ; enuncia e dimostra la complessità della ricerca. (07/09/23)
- **[7.3]** Definisci SA ; specifica lo pseudocodice per costruirlo via qsort; enuncia e commenta la complessità I/O worst-case della costruzione via qsort (e il caso in cui il massimo LCP è limitato da una costante c). (24/07/23, 07/09/21)
- **[7.4]** Mostra come CONTARE in $T[1, n]$ tutte le occorrenze di $P[1, p]$ con un Suffix Array in memoria; mostra e dimostra la complessità; qual è il costo I/O del RETRIEVAL delle posizioni se SA è su disco? (15/01/24, 10/02/21)
- **[7.5]** Descrivi come cercare $P[1, p]$ in un SA costruito su $T[1, n]$ e commenta la complessità in funzione di p e n . (15/07/25)
- **[7.6]** Dato il suffix tree di $T[1, n]$, come calcolare la sottostringa più lunga che occorre almeno L volte? Complessità in funzione di n . (12/06/20)

8 Integer Coding

Elenco testi — Elias–Fano e numero di bit

- **[8.1]** Data una sequenza di n interi non negativi $< u$, enuncia e dimostra lo spazio (in bit) della codifica di Elias–Fano. (27/05/24, 04/02/25)
- **[8.2]** Scrivi l’algoritmo per eseguire $\text{ACCESS}(i)$ su una sequenza codificata con Elias–Fano. (11/07/24)
- **[8.3]** Enuncia il numero di bit emessi dai codici γ , δ , Rice, Elias–Fano su interi/sequenze dati (esercizio ricorrente di calcolo). (vari appelli)

9 Statistical Coding

Elenco testi — Huffman, entropia, Arithmetic

- **[9.1]** Descrivi perché si introduce il Canonical Huffman; quali strutture dati tiene nel preambolo del file compresso; scrivi lo pseudocodice per decomprimere un simbolo. (09/02/24, 21/06/24, 15/07/25)
- **[9.2]** Dimostra l'upper bound in bit della codifica aritmetica in funzione dell'entropia e della lunghezza del testo. (09/12/24, 12/12/23)
- **[9.3]** Enuncia il numero di bit emessi da Huffman e Arithmetic dato un testo T di lunghezza n ed entropia empirica H ; commenta quando conviene l'uno o l'altro (anche in termini di tempo). (06/03/25, 03/06/25)
- **[9.4]** Dimostra che due testi T_1, T_2 , uno permutazione dell'altro, sono compressi con lo stesso numero di bit dalla codifica aritmetica. (23/06/25)
- **[9.5]** Perché la codifica aritmetica non usa l'estremo sinistro dell'intervallo finale ma parte dall'elemento centrale e lo tronca? (07/09/21)
- **[9.6]** (Huffman con entropia) Costruisci l'albero di Huffman per un alfabeto dato, calcola la lunghezza media del codeword e confrontala con l'entropia. (esercizio dispensa)

10 Strutture dati compresse

Elenco testi — Rank/Select succinti, FM-index, alberi succinti

- **[10.1]** Descrivi la struttura dati per supportare Rank su un array binario $B[1, n]$ in tempo $O(1)$ e valutane lo spazio. (16/01/23, 02/11/23)
- **[10.2]** Dato B , mostra la struttura per Rank e calcola lo spazio in bit. (05/06/23)
- **[10.3]** Dimostra la complessità in tempo e spazio (in bit) della soluzione per Rank in $O(1)$ e $o(m)$ bit su $B[1, m]$. (10/01/25, 04/02/25)
- **[10.4]** (Combinata) Dimostra l'upper bound in bit dell'Arithmetic coding; e mostra/dimostra lo spazio in bit della struttura succinta per Rank in $O(1)$. (12/12/23)

Part II

Risoluzioni dettagliate

11 Modelli & complessità I/O

– Bound I/O di sorting/permuting e disk striping

Quantità centrale. $\text{sort}(n) = \Theta\left(\frac{n}{B} \log_{M/B} \frac{n}{M}\right)$ I/O.

Multi-way MergeSort [1.1][1.3][1.5][1.6]. Si generano n/M run ordinati di lunghezza M ($O(n/B)$ I/O), poi si fondono a gruppi di $k = \frac{M}{B} - 1 = \Theta(M/B)$ run alla volta (un blocco in memoria per ciascun run + uno di output, minimo trovato con un min-heap di k elementi). Il numero di passate è $\log_k(n/M) = \log_{M/B}(n/M)$, ognuna a $O(n/B)$ I/O; totale $\text{sort}(n)$.

Permuting [1.2]. Costa $O(\min(n, \text{sort}(n)))$ I/O in alto e $\Omega\left(\min\left(\frac{n}{D}, \frac{n}{DB} \log_{M/B} \frac{n}{DB}\right)\right)$ in basso. *Conseguenza:* quando $B = \Omega(\log_{M/B}(n/M))$ (caso reale) i due bound coincidono: *ordinare e permutare costano uguale*, a differenza del modello RAM (n vs $n \log n$).

Disk striping [1.2][1.3][1.5][1.6]. Con D dischi in parallelo si tratta la i -esima “striscia” dei D dischi come un’unica pagina logica di $D \cdot B$ elementi (un solo I/O legge/scrive $D \cdot B$ elementi). Sostituendo $B \rightarrow DB$: $\Theta\left(\frac{n}{DB} \log_{M/(DB)} \frac{n}{M}\right)$ I/O.

Confronto con l’ottimo [1.4]. Il bound ottimo con D dischi è $\Omega\left(\frac{n}{DB} \log_{M/B} \frac{n}{DB}\right)$: il disk striping ha base del log $M/(DB)$ invece di M/B , quindi paga un fattore $\frac{\log(M/B)}{\log(M/(DB))}$, trascurabile se $DB \ll M$ (pochi dischi) ma non ottimale per D grande. Lo si usa per la sua semplicità.

12 Ordinamento

– Snow Plow: lunghezza attesa del run

Sia τ il numero di elementi processati in una fase (= lunghezza del run: ogni elemento estratto dall'heap viene scritto). All'inizio l'heap $\mathcal{H}$ ha M elementi. Ogni elemento letto dallo stream finisce nel buffer non ordinato $\mathcal{U}$ con probabilità p (è "troppo piccolo"), oppure resta in $\mathcal{H}$ (estende il run) con prob. $1 - p$. La fase finisce quando $\mathcal{H}$ si svuota e $\mathcal{U}$ si è riempito a M : quindi il numero atteso di elementi finiti in $\mathcal{U}$ è M , cioè $p \cdot \tau = M \Rightarrow \boxed{\tau = M/p}$.

- [2.1] $p = \frac{1}{2} \Rightarrow \tau = 2M$; $p = \frac{1}{4} \Rightarrow \tau = 4M$.
- [2.2] $p = \frac{1}{8} \Rightarrow \tau = 8M$.

Intuizione: più è raro che un elemento sia troppo piccolo (piccolo p), più a lungo l'heap resta pieno e più lungo è il run.

– LSD RadixSort: complessità e correttezza

Algoritmo. Per $i = L$ giù fino a 1, ordina (stabilmente, con CountingSort) le stringhe sulla i -esima cifra a partire dalla meno significativa. **Complessità** $O(L(n + \sigma))$ tempo, $O(N)$ spazio; per b bit e cifre da $r = \Theta(\log n)$ bit, $O(bn/\log n)$. **Correttezza (induzione + stabilità).** Invariante: dopo la fase sulla cifra i , le stringhe sono ordinate secondo le ultime i cifre. Base $i = 0$ banale. Passo: date α, β , se differiscono nella cifra i il CountingSort le ordina per quella cifra (la più significativa fra le ultime i); se hanno la stessa cifra i , la *stabilità* preserva l'ordine già acquisito per induzione sulle ultime $i - 1$ cifre. Dopo L fasi l'ordine è completo. La stabilità è essenziale: con un ordinamento non stabile l'invariante si romperebbe nei pareggi.

– String lower bound e Multikey QuickSort

Lower bound. Sia d_s il più corto prefisso che distingue s dalle altre e $d = \sum_s d_s$. Ogni algoritmo che ordina n stringhe fa $\Omega(d + n \log n)$ confronti di caratteri: d per distinguerle (bisogna leggerne i prefissi distinguenti), $n \log n$ per l'albero di decisione con $n!$ foglie. **Multikey QuickSort** (pseudocodice):

```
MultikeyQS(R,i):  if |R|<=1 return R; scegli pivot p; c=p[i]
R< = {s:s[i]<c}; R= = {s:s[i]=c}; R> = {s:s[i]>c}
return MultikeyQS(R<,i) ++ MultikeyQS(R=,i+1) ++ MultikeyQS(R>,i)
```

Sul ramo $R_=_$ si avanza a $i + 1$, sugli altri si resta a i . **Complessità** $O(d + n \log n)$ attesa: ogni stringa in $R_< \cup R_>$ contribuisce $O(\log n)$ confronti (come nella QuickSort classica: scende in un sottoinsieme $\leq \alpha|R|$); ogni permanenza in $R_=_$ avanza i , al più d_s volte. Somma $O(n \log n + d)$: *ottimo* per il lower bound. **RadixSort ottimo su stringhe binarie [2.3]:** raggruppando $\Theta(\log n)$ bit, $O(N \log \sigma / \log n)$.

Bounded QuickSort ($O(\log n)$ di stack)

Si elimina la ricorsione sul lato più grande (tail-recursion) e si ricorre solo sul lato più piccolo, garantendo profondità $\leq \log_2 n$.

```
BoundedQuickSort(S,i,j):
  while i<j:
    p = Partition(S,i,j)
    if (p-i) < (j-p): BoundedQuickSort(S,i,p-1); i = p+1
```

```
else:          BoundedQuickSort(S,p+1,j); j = p-1
```

La chiamata ricorsiva è sempre su $\leq n/2$ elementi $\Rightarrow$ dopo d chiamate annidate la taglia è $\leq n/2^d$, quindi $d \leq \log_2 n$: stack $O(\log n)$ nel caso peggiore, tempo $O(n \log n)$ atteso (pivot casuale) o garantito (mediana di campione).

13 Random Sampling

– Reservoir Sampling: pseudocodice e correttezza

```
riempi R[1..m] con i primi m elementi
for i=m+1,m+2,...: j = uniforme in [1,i]; if j<=m: R[j] = elemento
i-esimo
```

Costo $O(n)$, una passata, $O(m)$ spazio. **Correttezza** [3.1][3.2]. Fissato l'elemento in posizione t : entra con prob. m/t (se $t > m$) e sopravvive al passo $i > t$ con prob. $1 - \frac{1}{i} = \frac{i-1}{i}$. Quindi

$$\Pr[t \in R] = \frac{m}{t} \prod_{i=t+1}^n \frac{i-1}{i} = \frac{m}{t} \cdot \frac{t}{n} = \frac{m}{n},$$

per telescopio: campionamento uniforme, indipendente da t . (Con $m = 1$: prob. $1/n$.)

14 Set Intersection

Two-level intersection

A (lungo n) diviso in $h = n/L$ blocchi di taglia L ; array A' = primi elementi dei blocchi. *Fase 1*: merge di A' con B ($O(n/L + m)$), assegna ogni $b \in B$ al suo blocco. *Fase 2*: per ogni blocco selezionato, merge locale con i candidati ($O(mL + m)$). **Complessità** tempo $O(n/L + mL)$, minimizzato a $O(\sqrt{nm})$ con $L = \sqrt{n/m}$ ($f'(L) = -n/L^2 + m = 0$); I/O $O(\frac{n}{LB} + \frac{m}{B} + m)$. Batte sia il merge $O(n + m)$ (quando $m \ll n$) sia la binary search ripetuta $O(m \log n)$ (quando $m \approx n$).

Doubling / exponential search

Per ogni $b_j \in B$ si “galoppa” in A a salti $1, 2, 4, \dots$ dall’ultima posizione, poi binary search nell’ultimo intervallo. **Complessità** $O(m \log \frac{n}{m})$: sia $\Delta_j \leq 2(i_j - i_{j-1})$ la taglia del sotto-vettore per b_j ; costo $O(1 + \log \Delta_j)$; sommando e usando la concavità del log (Jensen) $\sum_j O(1 + \log \Delta_j) = O(m + m \log \frac{\sum \Delta_j}{m}) = O(m \log \frac{n}{m})$ poiché $\sum \Delta_j = O(n)$. Vantaggio sul mutual partition: accesso iterativo (miglior località).

Mutual partition; intersezione Elias–Fano

Mutual partition. Prendi la mediana p di B , cercala in A (binary search), stampala se presente, ricorri su $A[1, j] \cap B[1, \text{med} - 1]$ e $A[j+1, n] \cap B[\text{med}+1, m]$. Complessità $O(m \log \frac{n}{m})$ (ricorrenza $T(n, m) = O(\log n) + 2T(n/2, m/2)$), ottima ($\log \binom{n}{m}$). **Elias–Fano [4.6]**: si avanza alternando NEXTGEQ fra le due liste compresse: `y=B.NextGEQ(x); if y==x stampa; x=A.NextGEQ(y+ (y==x))`. Ogni ACCESS/NEXTGEQ è $O(1)$ ammortizzato (via Select sui bit alti); passi $O(m \log \frac{n}{m})$ come nel galloping, ma in spazio compresso (meno I/O).

15 Hashing, Skip List, Treap

– Hashing universale: definizione, esempio, prova

Definizione. $\mathcal{H} \subseteq \{h : U \rightarrow [0, m)\}$ è universale se $\forall x \neq y: |\{h : h(x) = h(y)\}| \leq |\mathcal{H}|/m$, cioè $\Pr_h[h(x) = h(y)] \leq 1/m$. **Esempio (prodotto scalare), m primo,** chiavi in base m $k = (k_0, \dots, k_{r-1})$: $h_a(k) = (\sum_i a_i k_i) \bmod m$, $|\mathcal{H}| = m^r$. **Prova.** Se $x \neq y$ differiscono nella cifra d ; la collisione equivale a $a_d(x_d - y_d) \equiv -\sum_{i \neq d} a_i(x_i - y_i) \pmod{m}$. Essendo m primo e $x_d - y_d \not\equiv 0$, $(x_d - y_d)$ è invertibile: per ogni scelta degli altri a_i (m^{r-1}) esiste *un solo* a_d che fa collidere. Quindi $\Pr[\text{coll}] = m^{r-1}/m^r = 1/m$: universale. [5.2] Con $m = O(n)$, la lista attesa è $\leq 1 + \alpha = 1 + n/m = O(1)$ (linearità dell'attesa su $\sum_{y \neq x} \Pr[h(x) = h(y)] \leq (n-1)/m$).

Cuckoo: teorema dei cammini c^{-L}/r

Enunciato. Per ogni coppia di celle i, j e $c > 1$, se $r \geq 2cn$, la probabilità che esista un cammino $i \rightarrow j$ di lunghezza L nel grafo cuckoo è $\leq c^{-L}/r$. **Prova (induzione su L).** Base $L = 1$: c'è un arco $i \rightarrow j$ se una delle n chiavi ha $\{h_1, h_2\} = \{i, j\}$, prob. $\leq n \cdot 2/r^2 \leq c^{-1}/r$ usando $r \geq 2cn$. Passo: un cammino di lunghezza L è uno di lunghezza $L-1$ da i a un ℓ più un arco $\ell \rightarrow j$; sommando su ℓ : $\sum_{\ell} \frac{c^{-(L-1)}}{r} \cdot \frac{2n}{r^2} = r \cdot \frac{c^{-(L-1)}}{r} \cdot \frac{1}{cr} = \frac{c^{-L}}{r}$. **Load factor.** $\Pr[i, j \text{ connessi}] = \sum_{L \geq 1} c^{-L}/r = \frac{1}{r(c-1)}$; la prob. di ciclo (fallimento) è $\leq \frac{1}{c-1}$; per tenerla $\leq \frac{1}{2}$ serve $c \geq 3$, cioè $r \geq 6n$, load factor $\leq 1/6$: il cuckoo classico garantisce $O(1)$ worst case solo a carichi bassi.

– Bloom Filter: 0-bit, falso positivo, 2 server

Bit-array $B[0, m-1]$, r funzioni hash. **Prob. bit a 0** dopo n inserimenti (rn accensioni): $p_0 = (1 - 1/m)^{rn} \approx e^{-rn/m}$. **Falso positivo:** servono tutti gli r bit di y a 1, $\Pr \approx (1 - p_0)^r = (1 - e^{-rn/m})^r$. **Ottimo** $r^* = \frac{m}{n} \ln 2$: allora $p_0 = \frac{1}{2}$ [5.4] ed errore $(\frac{1}{2})^{r^*} = 2^{-(m/n) \ln 2}$, esponenziale in m/n . **No falsi negativi** (i bit di una chiave inserita sono tutti 1). [5.5] operazioni: insert $O(r)$, search $O(r)$, niente delete. **Due server** [5.6]: il server di A manda il suo Bloom filter B_A (compatto) all'altro; il server di B testa ogni $b \in B$ contro B_A e restituisce i match; un solo round, con possibili falsi positivi (mai falsi negativi).

Skip List altezza $O(\log n)$; Treap

Skip List. Ogni elemento sale al livello ℓ con prob. $2^{-\ell}$. Il numero atteso di elementi sopra il livello $2 \log_2 n$ è $\leq n \cdot 2^{-2 \log_2 n} = 1/n$: per union bound l'altezza è $O(\log n)$ w.h.p. La ricerca, analizzata all'indietro, sale un livello in 2 passi attesi su $O(\log n)$ livelli $\Rightarrow O(\log n)$ atteso. **Treap** [5.8]. BST sulle chiavi + min-heap su priorità casuali: la struttura è univocamente determinata (radice = priorità minima), altezza attesa $O(\log n)$ (un nodo è antenato di un altro a distanza k di rango con prob. $\frac{1}{k+1}$, somma armonica). *Split* su chiave K non presente: inserisci $\langle K, -\infty \rangle$; per l'heap sale fino alla radice tramite rotazioni; a quel punto i suoi due sottoalberi sono esattamente $T_{<K}$ e $T_{>K}$ (rimuovi K). $O(\log n)$ atteso.

16 Prefix Search

– Patricia vs Compacted trie; storage

Compacted trie: archi etichettati da *sottostringhe* intere; in discesa confronta $O(p)$ caratteri; $O(n)$ nodi. **Patricia trie:** archi etichettati dal *solo primo carattere* + lunghezza al nodo; *blind search* (1 carattere per arco) e *un solo* accesso a disco alla stringa candidata per validare (Teorema della blind search: $O(1)$ I/O). **[6.1]** ricerca lessicografica: blind descent fino a una foglia, 1 accesso alla stringa per confronto completo, da cui la posizione lessicografica; $O(p)$ tempo, $O(1)$ I/O. **Uso nel two-level index [6.3]:** la Patricia indicizza in RAM solo le stringhe campione; la blind search dà il blocco in $O(p)$ tempo e $O(1)$ I/O, poi si scandisce/decomprime (front coding) il blocco su disco in $O(N_{occ}/B)$ I/O. **Storage [6.4]:** (1) array di puntatori ($N + (1 + w)n$ byte, ACCESS 1 I/O sparso); (2) stringhe contigue + offset (accesso sequenziale ai blocchi); (3) front coding a blocchi (spazio FC_B , ACCESS(i) decomprime dal capo del blocco). **[6.5]:** compacted trie con fan-out in hash table $\Rightarrow$ costruzione/ordinamento in $O(d \log \sigma)$ atteso, $O(d)$ spazio.

17 Substring Search

– Definizioni SA/LCP; ricerca e complessità

Suffix Array: $SA[1, n]$ = permutazione che ordina i suffissi, $suff_{SA[1]} < \dots < suff_{SA[n]}$ ($\Theta(n \log n)$ bit). **LCP array:** $lcp[i] = lcp(suff_{SA[i]}, suff_{SA[i+1]}) \in [0, n]$. **Ricerca di $P[1, p]$ [7.2]:** le occorrenze formano un intervallo $SA[i, j]$; due binary search per gli estremi. Complessità $O(p \log n)$ (confronto ingenuo) o $O(p + \log n)$ con gli array *Llcp/Rlcp* (a ogni passo o si dimezza l'intervallo senza leggere caratteri, $\leq \log n$ volte, o si avanza di ≥ 1 carattere di P , $\leq p$ volte).

COUNT e retrieval; costruzione qsort

COUNT [7.4][7.5]: individua l'intervallo $SA[i, j]$ con due binary search; $occ = j - i + 1$. Complessità $O(p \log n)$ (o $O(p + \log n)$ con LCP). **Retrieval su disco:** le posizioni $SA[i..j]$ sono *contigue* nell'array su disco, quindi si leggono in scansione sequenziale, $O(\frac{occ}{B})$ I/O (più $O(\frac{p}{B} \log n)$ per le binary search se anche T è su disco). **Costruzione via qsort [7.3]:** si ordinano gli indici $1..n$ con comparatore `strncmp` sui suffissi; worst case $O(n^2 \log n)$ tempo, I/O-inefficiente ($O(\frac{n}{B} \log n)$) perché (a) i confronti coinvolgono stringhe lunghe fino a $\Theta(n)$ e (b) l'ordinamento permuta i puntatori distruggendo la località. Se il massimo LCP $\leq c$ costante, ogni confronto legge $O(c)$ caratteri e il costo scende a $O(n \log n)$.

Suffix tree: sottostringa ripetuta $\geq L$ volte

Si annota ogni nodo interno con il numero di foglie del suo sottoalbero (= numero di occorrenze della stringa-cammino), con una DFS in $O(n)$. La sottostringa cercata corrisponde al nodo interno di *massima string-depth* il cui sottoalbero ha $\geq L$ foglie. Tempo totale $O(n)$ (costruzione del suffix tree in $O(n)$ + una DFS).

18 Integer Coding

– Elias–Fano: spazio (dimostrazione) e Access

n interi monotoni in $[0, u)$, $\ell = \lfloor \log_2(u/n) \rfloor$. *Bit bassi*: gli ℓ LSB in L ($n\ell$ bit). *Bit alti* $h_i = \lfloor s_i/2^\ell \rfloor$ in H : per ogni bucket v , tanti 0 quanti gli elementi con $h_i = v$, poi un 1; H ha n zeri e $u/2^\ell$ uni. **Spazio (prova)** [8.1][8.2]: $n\ell + (n + u/2^\ell)$; con $2^\ell > u/(2n)$ si ha $u/2^\ell < 2n$, quindi il totale è $\leq n\lfloor \log_2(u/n) \rfloor + 2n$ bit (a $\approx 2n$ bit dall'ottimo informativo $\log_2 \binom{u}{n}$). Il *conteggio esatto* da usare negli esercizi è $n\ell + n + u/2^\ell$. **Access**(j) [8.2]: $s_j = (\text{Select}_0(H, j) - j) \cdot 2^\ell + L[j]$ — i bit alti dalla posizione del j -esimo zero in H (via Select_0 , $O(1)$), concatenati con gli ℓ bit bassi. **Numero di bit dei codici** [8.3]: $|U(x)| = x$; $|\gamma(x)| = 2|B(x)| - 1$ (es. $\gamma(9) = 0001001$, 7 bit); $|\delta(x)| = |\gamma(|B(x)|)| + |B(x)|$ (es. $\delta(14) = 001001110$, 9 bit — $B(x)$ intero); $|R_k(x)| = \lfloor (x-1)/2^k \rfloor + k + 1$ (es. $R_4(83) = 10$ bit).

19 Statistical Coding

– Canonical Huffman; upper bound Arithmetic

Canonical Huffman [9.1]: introdotto per non memorizzare/navigare l'albero esplicito — usa le *stesse lunghezze* ottime ma codeword consecutive, decodificabili aritmeticamente con pochi cache miss. *Preambolo:* per ogni lunghezza ℓ , $\text{num}[\ell]$ e la prima codeword $\text{fc}[\ell]$ (regola $\text{fc}[\ell] = (\text{fc}[\ell - 1] + \text{num}[\ell - 1]) \cdot 2$), più la lista symb dei simboli ordinati per lunghezza. *Decodifica di un simbolo:* accumula bit aumentando ℓ finché $\text{code} \in [\text{fc}[\ell], \text{fc}[\ell] + \text{num}[\ell])$, poi restituisci $\text{symb}[\text{first}[\ell] + (\text{code} - \text{fc}[\ell])]$. **Arithmetic: upper bound [9.2]:** l'intervallo finale ha ampiezza $\prod_k \text{Pr}[s_k]$; i bit sono $\lceil \sum_k \log_2 \frac{1}{\text{Pr}[s_k]} \rceil + 1 \leq nH + 2$: overhead ≤ 2 bit totali.

Huffman vs Arithmetic; entropia

Bit emessi [9.3]: Huffman $n\mathcal{H} \leq L_{\text{tot}} < n\mathcal{H} + n$ (fino a 1 bit/simbolo di spreco, con $\mathcal{H} \leq L < \mathcal{H} + 1$); Arithmetic $\leq n\mathcal{H} + 2$. *Quando:* Arithmetic vince su sorgenti molto sbilanciate ($\mathcal{H} \ll 1$); Huffman è più veloce (canonico, niente moltiplicazioni). **Perché “dal centro” [9.5]:** l'estremo sinistro lo potrebbe richiedere infiniti bit per essere identificato univocamente; si sceglie un punto interno e lo si tronca alla precisione minima che lo mantiene in $[lo, hi)$, garantendo decodifica univoca con $\lceil \log_2 \frac{1}{hi-lo} \rceil + 1$ bit. **Huffman con entropia [9.6]:** costruisci l'albero greedy, $L = \sum_{\sigma} \text{Pr}[\sigma] \cdot \text{depth}(\sigma)$; es. $\Sigma = \{a..f\}$ con $\mathcal{P} = (.05, .10, .15, .30, .25, .15)$ dà $L = 2.45$ vs $\mathcal{H} \approx 2.39$, coerente con $\mathcal{H} \leq L < \mathcal{H} + 1$.

Arithmetic invariante per permutazione

Con modello statico basato sulle frequenze, l'ampiezza dell'intervallo finale è $\prod_{k=1}^n \text{Pr}[s_k] = \prod_{\sigma} \text{Pr}[\sigma]^{n_{\sigma}}$, che dipende solo dai *conteggi* n_{σ} dei simboli, non dal loro ordine. Due testi che sono uno la permutazione dell'altro hanno gli stessi conteggi $\Rightarrow$ stesso prodotto $\Rightarrow$ stesso numero di bit emessi $\lceil \sum_k \log_2 \frac{1}{\text{Pr}[s_k]} \rceil + 1$. (Vale per il modello statico.)

20 Strutture dati compresse

– Rank in $O(1)$ con $o(m)$ bit

Struttura a tre livelli su $B[1, m]$, con $Z = (\log m)^2$ e $z = \frac{1}{2} \log m$:

- *Big block* (taglia Z): rank assoluto all'inizio, $\frac{m}{Z} \log m = \frac{m}{\log m} = o(m)$ bit.
- *Small block* (taglia z): rank relativo al big block, $\leq Z$ quindi $O(\log \log m)$ bit ciascuno, $\frac{m}{z} O(\log \log m) = o(m)$ bit.
- *Lookup table* $R[b, o]$: #uni nei primi o bit di ogni configurazione da z bit; $2^z \cdot z \cdot O(\log z) = \sqrt{m} \cdot O(\dots) = o(m)$ bit.

$\text{Rank}_1(x) = r_{\text{big}} + r_{\text{small}} + R[B_{\text{small}}, o]$: tre accessi $O(1)$. **Spazio** $o(m)$ bit oltre a B ; **tempo** $O(1)$ worst case [10.1]–[10.4]. Punto chiave della prova: $z = \frac{1}{2} \log m$ rende la tabella $2^z = \sqrt{m}$ (sotto-lineare); $Z = (\log m)^2$ rende i rank relativi rappresentabili in $O(\log \log m)$ bit. [10.4] (combinata) include anche l'upper bound Arithmetic $\leq nH + 2$ (vedi [9.2]).

Nota su FM-index e alberi succinti

Se all'esame comparisse la *backward search* dell'FM-index: intervallo $[first, last]$ delle righe di $\mathcal{M}'$ prefissate da $P[i, p]$, aggiornato con $first = C[c] + \text{Rank}(c, first - 1)$, $last = C[c] + \text{Rank}(c, last) - 1$; correttezza dalla Proprietà 2 della BWT + LF-mapping; costo $O(p \cdot t_{\text{Rank}}) = O(p)$ se Rank è $O(1)$. Alberi succinti: $2n + o(n)$ bit, navigazione (padre/figli/foglia) con Rank/Select in $O(1)$.